ASSIGNMENT-12.4

2303A51064

B-16

Task 1 — Bubble Sort (Sorting Exam Scores)

Prompt:-

Write a Python program for Bubble Sort with comments explaining comparisons,swaps and iterations. Also explain time complexity.

Code:-

```python
def bubble_sort(scores):
    n = len(scores)
    for i in range(n):
        swapped = False
        for j in range(0, n-i-1):
            if scores[j] > scores[j+1]:
                scores[j], scores[j+1] = scores[j+1], scores[j]
                swapped = True
        if not swapped:
            break
    return scores
scores = [78, 45, 90, 62, 88]
print("Sorted Scores:", bubble_sort(scores))
```

```
Sorted Scores: [45, 62, 78, 88, 90]
```

Task 2 — Insertion Sort for Nearly Sorted Data

Prompt:-

Suggest a better sorting algorithm for nearly sorted data and implement Insertion Sort in Python with explanation.

Code:-

```python
def bubble_sort(arr):
    for i in range(len(arr)):
        for j in range(len(arr)-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

Insertion sort:-

```python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i-1
        while j >= 0 and key < arr[j]:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
data = [1, 2, 3, 5, 4]
print("Insertion Sort:", insertion_sort(data))
```

```
Insertion Sort: [1, 2, 3, 4, 5]
```

Task 3 — Linear Search vs Binary Search

Code1:-

```python
def linear_search(arr, key):
    """Search element in unsorted list"""
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
data = [15, 22, 9, 33, 40]
print("Linear Search:", linear_search(data, 33))
```

```
Linear Search: 3
```

Code2:-

```python
def binary_search(arr, key):
    """Search element in sorted list"""
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
data = [5, 10, 15, 20, 25]
print("Binary Search:", binary_search(data, 20))
```

```
Binary Search: 3
```

Task 4 — Quick Sort vs Merge Sort

Code1:-

```python
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
data = [8, 3, 5, 2, 9, 1]
print("Quick Sort:", quick_sort(data))
```

```
Quick Sort: [1, 2, 3, 5, 8, 9]
```

Code2:-

```python
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i=j=k=0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i+=1
            else:
                arr[k] = right[j]
                j+=1
            k+=1
        while i < len(left):
            arr[k] = left[i]
            i+=1
            k+=1
        while j < len(right):
            arr[k] = right[j]
            j+=1
            k+=1
    return arr
data = [8,3,5,2,9,1]
print("Merge Sort:", merge_sort(data))
```

```
Merge Sort: [1, 2, 3, 5, 8, 9]
```

Task 5 — Duplicate Detection Optimization

Code1:-

```python
def find_duplicates(arr):
    duplicates = []
    for i in range(len(arr)):
        for j in range(i+1, len(arr)):
            if arr[i] == arr[j]:
                duplicates.append(arr[i])
    return duplicates
data = [1,2,3,2,4,1]
print(find_duplicates(data))
```

```
[1, 2]
```

Code2:-

```python
def find_duplicates(arr):
    seen = set()
    duplicates = set()
    for num in arr:
        if num in seen:
            duplicates.add(num)
        else:
            seen.add(num)
    return list(duplicates)
data = [1,2,3,2,4,1]
print(find_duplicates(data))
```

```
[1, 2]
```